

Informe de Estrategia de Automatización QA – FrontEnd

Proyecto: SauceDemo (<https://www.saucedemo.com/>)

Stack: Playwright + Cucumber + Page Object Model

1. Objetivo

Diseñar e implementar una suite de pruebas automatizadas de extremo a extremo para la aplicación web SauceDemo, validando los flujos críticos de negocio: autenticación de usuarios, gestión del carrito de compras y proceso de checkout completo.

2. Alcance

La suite cubre los siguientes flujos funcionales:

- Login exitoso con usuario estándar (standard_user)
- Login fallido con usuario bloqueado (locked_out_user)
- Login fallido con credenciales inválidas
- Agregar un producto al carrito desde el inventario
- Verificar productos listados en el carrito
- Completar el proceso de compra hasta la confirmación de pedido

3. Stack Tecnológico

Herramienta	Rol en el proyecto
Playwright	Automatización del navegador
@cucumber/cucumber	Framework BDD / runner de tests
Node.js	Entorno de ejecución
Chromium	Navegador de ejecución

4. Patrón de Diseño: Page Object Model (POM)

Se implementó el patrón Page Object Model (POM) como estrategia principal de organización del código. Este patrón encapsula la estructura y los comportamientos de cada página de la aplicación en una clase dedicada, separando la lógica de interacción con el navegador de la lógica de los tests.

4.1 Beneficios aplicados

- Mantenibilidad: si cambia un selector, solo se actualiza en un lugar (la clase Page Object).
- Legibilidad: los step definitions son limpios y expresivos, sin selectores visibles.
- Reutilización: los métodos de las páginas son compartidos entre múltiples escenarios.
- Escalabilidad: agregar nuevas páginas o acciones no impacta los tests existentes.

4.2 Estructura de Page Objects

Clase	Página que representa	Responsabilidades principales
LoginPage.js	/ (página raíz)	Ingresa credenciales, hacer click en Login, leer mensaje de error
InventoryPage.js	/inventory.html	Agregar productos al carrito, leer badge del carrito, ir al carrito, verificar logo
CartPage.js	/cart.html	Listar productos en carrito, hacer click en Checkout
CheckoutPage.js	/checkout-step-one/two.html	Ingresa datos personales, continuar, finalizar compra, leer confirmación

5. Estrategia BDD con Cucumber y Gherkin

Se adoptó el enfoque Behavior-Driven Development (BDD) mediante Cucumber, lo que permite que los escenarios de prueba sean comprensibles tanto para el equipo técnico como para stakeholders de negocio sin conocimiento de programación.

5.2 Feature files

Feature file	Escenarios	Tipo de usuario
login.feature	Login exitoso, login bloqueado, credenciales inválidas	standard_user, locked_out_user, inválido
compra.feature	Agregar al carrito, ver carrito, flujo completo de compra	standard_user

6. Estrategia de Selectores

Se priorizó el uso de atributos data-test provistos por la propia aplicación SauceDemo. Estos atributos son más robustos que clases CSS o XPath

7. Estrategia de Validaciones

Escenario	Validación aplicada	Elemento validado
Login exitoso	URL + contenido visual	URL /inventory.html + logo 'Swag Labs'
Login bloqueado / inválido	Mensaje de error	[data-test="error"] con texto exacto

Agregar al carrito	Badge del carrito	[data-test="shopping-cart-badge"] = '1'
Ver carrito	Nombre del producto	[data-test="inventory-item-name"] contiene nombre
Compra completada	Mensaje de confirmación	[data-test="complete-header"] = 'Thank you for your order!'

8. Estructura del Proyecto

El proyecto sigue una organización clara por capas de responsabilidad:

Carpeta / Archivo	Propósito
features/	Escenarios de prueba en lenguaje Gherkin (negocio)
pages/	Page Objects — encapsulan selectores y acciones por página
step-definitions/	Implementación de los pasos Gherkin, conectan features con pages
support/world.js	Configura el navegador (Before/After) y el timeout global de 30s
cucumber.js	Configuración del runner de Cucumber (paths, require, formato)
reports/	Reporte HTML generado automáticamente tras cada ejecución

9. Conclusiones

La combinación de Playwright + Cucumber + POM proporciona una suite de automatización mantenible, legible y escalable:

- Playwright ofrece APIs modernas, esperas automáticas y soporte multi-navegador.
- Cucumber acerca los tests al lenguaje de negocio, facilitando revisiones y nuevos escenarios.
- POM centraliza los selectores, reduciendo el impacto de cambios en la UI.
- El uso de data-test como estrategia de selección garantiza estabilidad ante cambios visuales.